



Daniil Shchennikov

EMOTION RECOGNITION FROM TEXT AND VOICE USING AI

AI-based system

for emotion recognition from both spoken and written
input

Technology

2025

ABSTRACT

Author	Daniil Shchennikov
Title	Emotion recognition from text and voice using AI AI-based system for emotion recognition from both spoken and written input
Year	2025
Language	English
Pages	54 + 1 Appendices
Name of Supervisor	Johan Dams

This thesis investigates emotion recognition across two modalities—text and speech—by building a privacy-first, reproducible prototype and comparing modality-appropriate baselines under both formal tests and informal mobile trials. The text pathway fine-tunes BERT-base on the GoEmotions dataset as a multi-label classifier with sigmoid/BCE and per-class thresholding calibrated on validation data. The speech pathway trains a compact CRDNN using SpeechBrain on an acted-speech corpus as a single-label classifier with softmax/NLL. A React Native/Expo client and FastAPI backend deliver predictions to a phone and capture star-rating feedback; privacy is enforced by storing only hash which is privacy safe-finger print of text and audio, and not storing raw data.

On test splits derived from the datasets, the audio model reaches 0.833 accuracy (0.829 macro-F1) across eight classes, while the text model achieves 0.566 micro-F1, 0.52 macro-F1, and 0.409 subset accuracy over 28 labels. In informal, in-the-wild trials, audio performance declines to the mid-60% range, whereas text remains comparatively stable but exhibits recurring confusions (e.g. Neutral vs. low-arousal phrasing). A conceptual mapping aggregates fine-grained text labels to rough audio categories to support qualitative, analysis-only cross-modal comparison. Prototype average latency is practical for interactive use (≈ 458 ms text; ≈ 500 ms audio).

Contributions include: (1) modality-aware baselines with calibrated decision rules; (2) a transparent, end-to-end pipeline and mobile prototype that unify formal and informal evaluation; (3) empirical evidence clarifying when each modality excels (prosody-driven vs. lexically explicit affect); and (4) actionable guidance for future work—speaker-disjoint, spontaneous speech data; augmentation; automated threshold refresh; late-fusion baselines; and on-device optimization. Overall, the results argue for modality-aware integration of affect models into larger systems where each stream contributes its strengths while respecting privacy.

Keywords emotion recognition, BERT, GoEmotions, RAVDESS, SpeechBrain, CRDNN, multi-label classification, speech emotion recognition, threshold tuning, domain shift, mobile prototype

CONTENTS

ABSTRACT	2
1 INTRODUCTION	9
1.1 Motivation and context	9
1.2 Use of AI in the thesis	10
1.3 Research Scope and Problem Definition	11
1.4 Prototype and Research Design	11
1.5 Findings, Contributions, and Limitations	13
2 BACKGROUND AND THEORETICAL FRAMEWORK.....	14
2.1 What “emotion recognition” means in this thesis.....	14
2.2 Modality comparisons in prior work.....	15
2.3 Affective computing and why modality matters.....	16
2.4 From lexicons to Transformers	17
2.5 GoEmotions	18
2.6 Speech emotion recognition (SER).....	19
2.7 Modality-aware learning objectives and decision rules.....	21
2.7.1 Text as multi-label	21
2.7.2 Audio as single-label.....	23
2.8 Cross-modal alignment.....	23
2.9 Reliability, validity, and bias considerations.....	24
2.10 Adopted conceptual framework.....	25
3 RESEARCH METHODOLOGY AND APPROACH	26
3.1 Datasets and label spaces	26
3.2 Preprocessing pipelines.....	29
3.3 Models and objectives	30
3.4 Training configuration and schedules	32
3.5 Inference and exported artifacts.....	32
3.6 Evaluation protocol and metrics	33
3.7 Reproducibility and implementation notes.....	34
4 IMPLEMENTATION AND SYSTEM DESIGN.....	35
4.1 Design goals and constraints	35
4.2 System architecture (high-level)	35

4.3	Data flow from phone to model and back.....	36
4.4	Backend implementation.....	37
4.5	API surface and validation.....	38
4.6	Mobile client	38
4.7	Persistence, privacy, and analytics	39
4.8	Runtime environment and deployment.....	40
4.9	System Efficiency and Constraints	41
5	RESULTS	42
5.1	Evaluation setup (recap).....	42
5.2	Text modality — formal test results	42
5.3	Audio modality — formal test results.....	44
5.4	Cross-modal comparison (rough mapping).....	44
5.5	Informal prototype trials (in-app).....	45
5.6	Sensitivity	46
5.7	Error analysis and threats to validity.....	47
5.8	Summary	48
6	DISCUSSION AND CONCLUSION.....	49
6.1	Interpreting the findings	49
6.2	Where errors come from	50
6.3	Validity, reliability, and ethics	50
6.4	Reflections and Future Work.....	51
6.5	Conclusion.....	52
	REFERENCES.....	53
	APPENDIX: SOURCE CODE (GITHUB)	55

FIGURES

Figure 2.1 BERT pre-training and fine-tuning schematic. Adapted from Devlin et al. (2019, pp. 4171–4173).	18
Figure 2.2 GoEmotions baseline per-class P/R/F1 (BERT-base), reproduced from Demszky et al. (2020, p. 4047).	19
Figure 2.3 The selected per-class thresholds (τ) tuned on validation, alongside the global τ and probability floor.	22
Figure 3.1 GoEmotions test samples with multi-label annotations (comment text with label index lists).	27
Figure 3.2 RAVDESS test split preview (ID, wav, duration, emotion). ..	28
Figure 3.3 Canonical label order for text (28 labels).	28
Figure 3.4 Canonical label order for audio (8 labels).	28
Figure 3.5 Text tokenization and batching configuration (bert-base-uncased, max length 128, dynamic padding).	29
Figure 3.6 Audio feature extractor (log-Mel: n_fft=1024, hop=256, n_mels=80) and batch padding.	30
Figure 3.7 Weighted BCE implementation for multi-label text: pos_weight computation and custom trainer hook.	31
Figure 3.8 Audio dataset and dataloader setup (signal/label pipelines; batch collation).	31
Figure 3.9 Post-training probability computation via sigmoid for validation/test logits.	33
Figure 3.10 Per-class decision thresholds for text (global τ and probability floor shown).	33
Figure 4.1 System diagram.	36
Figure 4.2 Feedback API endpoint example.	37
Figure 4.3 Two examples of API endpoints: (a) text; (b) audio	38
Figure 4.4 Mobile application screens: a) Home; b) Audio; c) Text; d) Feedback	39
Figure 4.5 Audio fine-tuning on CPU.	40
Figure 4.6 Audio fine-tuning on T4 GPU.	40
Figure 5.1 Text training dashboard / improved run.	43
Figure 5.2 Text baseline run.	43

Figure 5.3 Per-class thresholds.....	43
Figure 5.4 Confusion matrices.	44
Figure 5.5 App analytics: a) audio; b) text; c) text vs audio.	46
Figure 5.6 Audio training/testing on CPU.....	47
Figure 5.7 Audio training/testing on T4 GPU.	47

ABBREVIATIONS

API	Application Programming Interface
ASR	Automatic Speech Recognition
BCE	Binary Cross-Entropy
BERT	Bidirectional Encoder Representations from Transformers
BiGRU	Bidirectional Gated Recurrent Unit
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CMU-MOSEI	Carnegie Mellon University – Multimodal Opinion Sentiment and Emotion Intensity
CRDNN	Convolutional Recurrent Deep Neural Network
CLS	Classification
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
cuDNN	CUDA Deep Neural Network Library
dB	Decibel
FFT	Fast Fourier Transform
F1	F1-Score (harmonic mean of Precision & Recall)
FP16	Half-Precision 16-bit Floating Point
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation

LR	Learning Rate
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
MLM	Masked Language Modeling
MLP	Multi-Layer Perceptron
MOSI	Multimodal Opinion-level Sentiment Intensity
MOSEI	Multimodal Opinion Sentiment and Emotion Intensity
NLL	Negative Log-Likelihood
NLP	Natural Language Processing
PII	Personally Identifiable Information
RAVDESS	Ryerson Audio-Visual Database of Emotional Speech and Song
ReLU	Rectified Linear Unit
REST	Representational State Transfer
RNN	Recurrent Neural Network
IEMOCAP	Interactive Emotional Dyadic Motion Capture
SER	Speech Emotion Recognition
SHA-256	Secure Hash Algorithm 256-bit
SQL	Structured Query Language
SQLite	Lightweight SQL Database Engine
STFT	Short-Time Fourier Transform
UI	User Interface
UUID	Universally Unique Identifier
UX	User Experience
WSL	Windows Subsystem for Linux

1 INTRODUCTION

This thesis builds a privacy-first, two-modal emotion-recognition prototype and compares text (BERT fine-tuned on GoEmotions) with speech (a compact CRDNN via SpeechBrain) under both formal tests and informal mobile use. The practical goal is not to “solve” affect understanding but to establish modality-aware baselines, observe how they behave in real world cases, and identify the trade-offs that matter for real applications. But first, few words about how the author of this thesis came to the topic and the reason behind the interest.

1.1 Motivation and context

Emotion-aware systems are increasingly integrated into everyday tools—from conversational assistants and wellbeing apps to learning platforms and accessibility technologies. Yet, most deployed systems still rely on a single modality, usually text, or treat audio cues in isolation. This thesis investigates a practical, two-modal approach—text and voice—to emotion recognition, implemented as a working mobile prototype that accepts user input either audio or text, returns predicted emotions, and captures star-rating plus comment feedback to measure correctness both modalities. The goal is not only to compare the two modalities under controlled conditions but also to observe how they behave in informal, real-world use.

Technically, the text pathway builds on BERT—a bidirectional Transformer pre-trained via masked-language modeling and then fine-tuned for downstream tasks—chosen for its strong transfer performance across a wide suite of NLP benchmarks and minimal task-specific customization needs. On the data side, the text model is fine-tuned on GoEmotions, a 58k-example dataset labeled with 27 fine-grained emotions plus Neutral, making it particularly suitable for nuanced, multi-label prediction. For voice, the system leverages SpeechBrain, an open, PyTorch-native toolkit that provides research-oriented recipes across

many speech tasks, including emotion recognition; it is designed for modularity, reproducibility, and swift experimentation. The audio model is evaluated on RAVDESS (categorical vocal emotions), as planned in the project plan.

1.2 Use of AI in the thesis

In this thesis, artificial intelligence (AI) tool such as ChatGPT has been utilised to support the research and writing process. AI was used for searching relevant information and faster documentation read through, generating a few small code snippets, creating graph based on the data obtained after the implementation phase, and checking the language of the written text.

I have edited all AI-generated content to ensure that it accurately reflects my original intent and that the scientific content remains authentic and verifiable. The figure generated with AI (Figure 2.3) is based on real data. AI was used to go through documentations faster. For instance, I asked about what arguments certain external module functions require and what these functions return. And I must state that sometimes AI managed to help me and sometimes it gave me non-existent functions' explanations, in this case I read documentation. Additionally, ChatGPT generated a script which is called `plot_confusion_matrix.py`.

I have ensured the authenticity of all content and respected copyrights throughout the work. Any new ideas or information suggested by AI have been verified using original academic sources. All sources listed in the references have been personally reviewed.

For language improvement and proofreading, I used GhatGPT and Google Translate to enhance readability and grammatical accuracy while retaining my own writing style. AI application was used responsibly, with attention to data protection and academic integrity.

1.3 Research Scope and Problem Definition

This thesis studies how reliably two machine learning models— a multi-label text classifier and a single-label speech classifier—can recognize human emotions and how their behaviors differ in two contexts: formal evaluations on test splits of benchmark datasets and informal trials conducted through a mobile prototype. The scope is intentionally narrow to ensure methodological clarity. Only two modalities are explored, text and speech, excluding visual data. The speech branch is modeled as a categorical 8 classes, while the text branch adopts a fine-grained 28-label taxonomy (27 emotions plus Neutral). The prototype is a research instrument designed to explore emotion recognition performance, rather than as a clinical or safety-critical tool.

As stated above the two modalities are explored and the choice of modalities reflects their complementary nature: text conveys lexical and semantic cues, while speech captures prosody and tone. BERT was selected for text emotion recognition because of its proven ability to transfer effectively with minimal task-specific modification, while the GoEmotions dataset provides a curated, fine-grained, and multi-label supervision. For speech emotion recognition, SpeechBrain was chosen for its transparent, recipe-driven SER pipelines, and an acted-speech corpus was used to establish a clean single-label baseline. This pairing makes differences explicit rather than architectural.

1.4 Prototype and Research Design

The prototype consists of a React Native/Expo mobile application connected to a FastAPI backend. The app enables users to submit either textual or audio input, which the backend processes through two separate adapters. The text adapter tokenizes the input, performs inference using the fine-tuned BERT model, converts logits to probabilities, and applies calibrated, per-class thresholds saved after validation. The audio adapter converts user input to WAV format, extracts log-Mel features,

performs inference using a CRDNN model, and returns a softmax label with scores. The app displays predictions to the user and collects optional star-rating and comment feedback, which is stored locally in SQLite. For privacy reasons, only SHA-256 hashed fingerprints of inputs are saved—no raw audio or text data is stored.

The prototype design described above is chosen to help to implement the research which is guided by the following questions:

- How accurately can a fine-tuned BERT model recognize emotions from written input?
- How accurately can a SpeechBrain-based model recognize emotions from spoken input?
- Which modality demonstrates greater consistency across emotion classes and usage contexts?

Before finding answers on the above questions, it is hypothesized that the text-based model will be more robust to real-world inputs than the acted-speech-based audio model, whose performance may decline in everyday conditions meaning outside controlled environments.

To explore these questions, both models were implemented using standard, reproducible deep learning practices. The text classifier uses BERT with sigmoid activations and binary cross-entropy loss (BCE). Two configurations were tested, with the final model having class-weighted BCE, early stopping, and per-class thresholding. The audio model uses SpeechBrain's CRDNN with softmax and negative log-likelihood loss (NLL) over eight emotion classes. To enable cross-modal analysis, fine-grained text labels were conceptually mapped into the eight broader audio categories, allowing comparative insights without retraining either model.

1.5 Findings, Contributions, and Limitations

Preliminary results show that on formal test splits, the audio model achieves 0.833 accuracy and 0.829 macro-F1, while the text model reaches 0.566 micro-F1, 0.520 macro-F1, and 0.409 subset accuracy. In informal app-based trials, the audio model's accuracy decreases to the approximately 60% range due to domain shift from acted to spontaneous speech, while the text model remains comparatively stable but confuses low-arousal content with Neutral and struggles with Disgust. These observations highlight the need for calibrated decision rules in text emotion recognition and improved domain robustness in speech-based models.

Based on previously described prototype and research design as well as the preliminary results, the thesis contributes several outcomes:

- A working, privacy-first prototype (mobile + backend) that makes formal and informal evaluation seamless.
- Fully reproducible training and inference setups, including exported thresholds and per-class metrics.
- A comparative, modality-aware analysis of text versus speech emotion recognition.
- Practical recommendations for threshold calibration in text classification and for bridging the lab-to-field performance gap in speech emotion recognition.

Certain limitations must be acknowledged. Comparisons involve different labeling schemes—multi-label text versus single-label audio—and rely on a mapping that inevitably compresses nuance. Acted-speech benchmarks may overestimate field performance; informal trials document the gap. During testing, no raw text or audio was stored, and any potential deployment would require explicit consent, data minimization, and bias auditing to ensure compliance with ethical and privacy standards.

2 BACKGROUND AND THEORETICAL FRAMEWORK

This chapter establishes the conceptual ground for the thesis and motivates the modeling choices used later. It begins by clarifying how “emotion recognition” is framed here and why categorical labeling fits the two datasets under study. It then views how affective information is distributed across text and speech, drawing on prior multimodal work to explain why text often provides the strongest unimodal signal while audio contributes prosodic cues that help on specific categories. With that context, the chapter reviews core methods for text emotion recognition (from lexicons to Transformers) and for speech emotion recognition (SER) using modern toolkits. Finally, it specifies the modality-aware learning objectives used in the thesis (multi-label text vs. single-label audio) and introduces the rough mapping that enables fair, analysis-only comparisons across modalities. The next section (2.1) starts with a precise definition of “emotion recognition” as used in this work and sets the stage for the rest of the framework.

2.1 What “emotion recognition” means in this thesis

Emotion recognition, as used here, is the automatic assignment of emotion labels to human inputs in text or speech. Two complementary framing choices shape the rest of the work. To situate the modeling decisions that follow, the discussion first clarifies how “emotion” is represented in this thesis and why that choice fits the data at hand.

Categorical vs. dimensional views. Psychology offers both discrete categories (anger, joy, fear) and continuous dimensions (valence, arousal). This thesis adopts categorical labels because both target datasets—GoEmotions (text) and an acted-speech corpus used via SpeechBrain recipes (audio)—are categorical by design. GoEmotions is multi-label—“...58k English Reddit comments, labeled for 27 emotion categories or Neutral,” (Demszky et al., 2020, p. 4040)—meaning one text can express several emotions simultaneously, and the baseline they report

shows “average F1-score of 0.46” for a BERT-base fine-tune (Demszky et al., 2020, p. 4041), which motivates further work on calibration and class imbalance. Whereas the speech corpus is single-label per input. This asymmetry is central to the modeling choices that follow. In turn, acknowledging this asymmetry sets up how the thesis defines a fair basis for comparing the two modalities.

In summary, building on the categorical framing and the modality asymmetry, the aim is not to solve general affect understanding, but to build two strong, modality-appropriate baselines that can be embedded into a practical prototype and compared fairly. Accordingly, the evaluation focuses on metrics and procedures that respect each dataset’s labeling schema while still allowing a coherent, side-by-side assessment of performance in a real-world context.

A consistent pattern in multimodal affect research is that text is typically the strongest unimodal signal, while audio contributes complementary prosody that helps on specific categories. As one survey/baseline paper concludes: “The text modality plays the most important role ... [and] the fusion of the modalities shows more impact” for emotion tasks. (Poria et al., 2018)

2.2 Modality comparisons in prior work

Across MOSI/MOSEI/IEMOCAP-style benchmarks, studies repeatedly find text dominates unimodal performance, with audio providing gains when fused—especially for high-arousal or prosody-heavy emotions. Multimodal sentiment analysis studies show that, in speaker-inclusive settings, “audio modality consistently performs better than visual,” yet “the text modality plays the most important role,” and fusion improves over unimodal baselines ((Poria et al., 2018). In CMU-MOSEI, the dataset was created specifically to analyze how modalities interact—“the largest dataset ... for multimodal sentiment analysis and emotion recognition”—and their fusion model is designed to dynamically weigh

streams: “DFG is able [to] choose which multimodal dynamics to rely on.” (Zadeh et al., 2018)

Two practical cautions emerge. First, fusion gains depend on dataset conditions: when language already carries strong affective cues, adding audio yields modest but consistent improvements; when text is sparse or ambiguous, audio helps more. Second, fusion should not over-weight weak modalities: recent work warns that treating modalities uniformly “may lead to underestimating... strong modalities while overemphasizing... weak ones,” motivating text-oriented attention when text is the reliable carrier of semantics. (Quan et al., 2024)

Finally, papers focused exactly on speech–text show the combined system “deliver[s] more accuracy by using speech and text data,” leveraging each stream’s strengths. (Byun, 2021)

These findings justify the decision to keep modality-aware objectives (multi-label text vs. single-label audio), report unimodal baselines first, and use a coarse mapping only for comparison, not training.

2.3 Affective computing and why modality matters

Affective computing studies how machines can sense and respond to human emotions. Text and speech carry different signals, and appreciating this contrast is the starting point for the modeling choices in this thesis. Framing the discussion around what each modality can and cannot express helps justify the distinct objectives that are adopted later.

On the textual side, text encodes lexical and syntactic cues (choice of words, intensifiers, negation, pragmatic markers, emojis), but lacks cues that are conveyed with audio. In practice, these surface and contextual markers enable fine-grained categorical judgments, yet the absence of sound or timing means textual emotion recognition must infer affect without many of the cues humans rely on; this limitation motivates complementary treatment of speech.

Conversely, speech encodes acoustics (pitch, energy, timbre, rhythm, speaking rate, pauses), including subtle prosodic contours that are often decisive for emotion, but it lacks the explicit semantics of written words. As a result, audio models excel at capturing how something is said rather than precisely what is said, which makes them powerful counterparts to text models while also complicating direct, apples-to-apples comparisons.

Because each modality drops some information the other carries, treating them with modality-aware objectives and losses (Section 2.6) is not just an implementation detail—it is a theoretical commitment that respects how the signals differ. This commitment also sets up the evaluation choices that follow, where metrics and thresholds are matched to each modality’s signal properties before any cross-modality comparison is attempted.

2.4 From lexicons to Transformers

Early systems combined emotion lexicons with linear models. Deep learning then lifted performance via CNN/RNN encoders, but the major step-change came with Transformer pretraining. Modern NLP benefits from Transformer pretraining where “deep bidirectional representations [are learned] by jointly conditioning on both left and right context” (Devlin et al., 2019, p. 4171). The pre-trained BERT “can be fine-tuned with just one additional output layer” (Devlin et al., 2019, p. 4171), which keeps downstream modeling simple and reproducible. Simple fine-tuning is enabled by MLM. In practice, a single [CLS] representation plus a task head often suffices, yielding the most modern results across classification benchmarks with minimal architecture changes. In multi-modal studies, this strong text backbone often becomes the anchor that fusion models rely on, with audio offering incremental gains where prosody matters. (Multimodal Language Analysis in the Wild: CMU-MOSEI Dataset and Interpretable Dynamic Fusion Graph; AmirAli Bagher Zahedi; 2018)

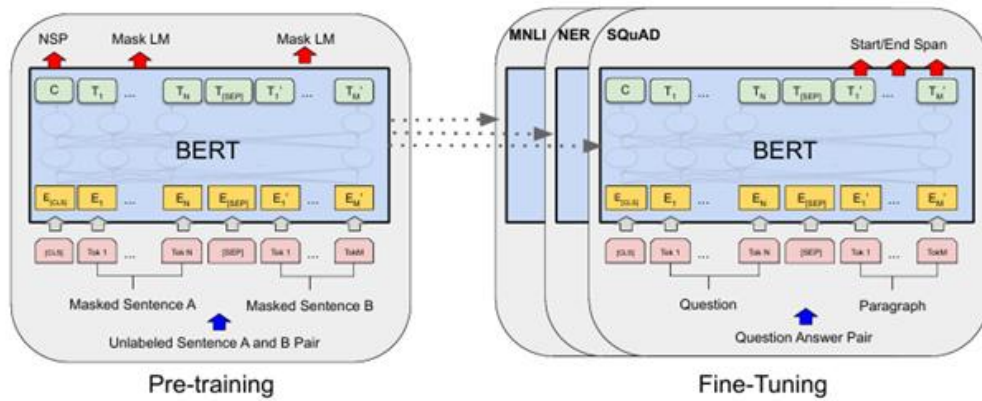


Figure 2.1 BERT pre-training and fine-tuning schematic. Adapted from Devlin et al. (2019, pp. 4171–4173).

2.5 GoEmotions

GoEmotions provides approximately 58k carefully curated Reddit comments annotated with 27 emotions + Neutral, with multi-label ground truth and an explicit discussion of annotation quality and dataset biases. For downstream use, authors report a BERT-base baseline F1 of ~ 0.46 (p. 4041) over the fine-grained taxonomy—leaving room for improvement and encouraging method work such as class-imbalance handling and threshold tuning. The dataset also supports rougher groupings (e.g., Ekman-style) when needed.

Emotion	Precision	Recall	F1
admiration	0.53	0.83	0.65
amusement	0.70	0.94	0.80
anger	0.36	0.66	0.47
annoyance	0.24	0.63	0.34
approval	0.26	0.57	0.36
caring	0.30	0.56	0.39
confusion	0.24	0.76	0.37
curiosity	0.40	0.84	0.54
desire	0.43	0.59	0.49
disappointment	0.19	0.52	0.28
disapproval	0.29	0.61	0.39
disgust	0.34	0.66	0.45
embarrassment	0.39	0.49	0.43
excitement	0.26	0.52	0.34
fear	0.46	0.85	0.60
gratitude	0.79	0.95	0.86
grief	0.00	0.00	0.00
joy	0.39	0.73	0.51
love	0.68	0.92	0.78
nervousness	0.28	0.48	0.35
neutral	0.56	0.84	0.68
optimism	0.41	0.69	0.51
pride	0.67	0.25	0.36
realization	0.16	0.29	0.21
relief	0.50	0.09	0.15
remorse	0.53	0.88	0.66
sadness	0.38	0.71	0.49
surprise	0.40	0.66	0.50
macro-average	0.40	0.63	0.46
std	0.18	0.24	0.19

Table 4: Results based on GoEmotions taxonomy.

Figure 2.2 GoEmotions baseline per-class P/R/F1 (BERT-base), reproduced from Demszky et al. (2020, p. 4047).

Importantly, the paper (Demszky et al., 2020, p. 4042) documents known limitations and biases (domain = Reddit; annotator demographics; content filtering), which motivates caution when generalizing to other populations or registers—an ethical point revisited later in the thesis.

2.6 Speech emotion recognition (SER)

SER estimates a categorical emotion from an input. Classic features include MFCCs, chroma, mel-spectrograms, and pitch/energy statistics; modern systems frequently use CNNs, RNNs/transformers, or hybrids over time–frequency representations. This thesis does not claim novelty in feature engineering; instead, it leverages standard practices exposed by modern SER toolkits. Against this backdrop, the choice of implementation framework becomes central to ensuring comparability and reproducibility across experiments.

SpeechBrain is a research-oriented, PyTorch-native toolkit that provides recipes, pretrained models, and a clean experiment structure spanning many speech tasks (Ravanelli et al., 2021, p. 1) (ASR, speaker recognition, SLU, enhancement, and classification tasks like SER). Its design

emphasizes modularity, transparency, and reproducibility, which aligns well with a thesis setting that values shareable configurations and logs. By grounding the experiments in a well-established toolkit, the focus can shift from plumbing details to disciplined evaluation on a representative SER corpus. Comparative studies on MOSEI explicitly frame analysis around inter- and cross-modal interactions, using architectures that can select which stream to trust per instance—e.g., dynamic fusion mechanisms (Multimodal Language Analysis in the Wild: CMU-MOSEI Dataset and Interpretable Dynamic Fusion Graph). At the same time, newer text-oriented fusion methods caution against over-emphasizing noisy audio when text is already decisive. (TCAN: Text-oriented Cross Attention Network for Multimodal Sentiment Analysis)

The acted-speech dataset used for audio experiments is well known in SER literature; it supplies categorical labels for a fixed set of emotions and controlled recordings by trained performers. The project plan specifies its use as the target audio benchmark. Therefore, model choices and hyperparameters are tuned with the dataset’s constraints in mind, preparing the ground for clear baselines and later comparisons to text-based results.

The RAVDESS-style corpus used here provides eight categorical emotions recorded by trained performers in a controlled setting: “The RAVDESS dataset... consists of speech and song records... into eight separate levels... [with] 24 trained performers, 12 males and 12 females” (Puri et al., 2022, J. Healthcare Eng., §2.1 Data Description). (Note: this article has been retracted; it is cited only for public dataset description that aligns with the original RAVDESS paper.) While the corpus simplifies some aspects of real-world variability, its careful control and clear labeling make it suitable for establishing modality-appropriate baselines that can later be stress-tested under informal, real-usage conditions.

2.7 Modality-aware learning objectives and decision rules

This thesis adopts different formulations for text and audio to match their label spaces and data properties. With that design principle in mind, the text pathway is framed explicitly as a multi-label problem, while the audio pathway remains single-label; the sections below focus on the former and how its training/inference choices support fair comparison later.

2.7.1 Text as multi-label

Each text may express multiple emotions. The output layer uses independent sigmoid activations per class and optimizes BCE. Thresholding is crucial with GoEmotions’ multi-label ground truth—“...labeled for one or more of 27 emotions or Neutral,” (Demszky et al., 2020, p. 4040)—so per-class threshold tuning is adopted with a probability floor, and the final thresholds are exported for reproducibility. This establishes a clear separation between model training and decision calibration, which is important when classes differ widely in frequency.

Two fine-tuning strategies were used. Approach 1 (baseline): BCE with uniform per-class weighting and a fixed threshold at 0.5 for inference. Approach 2 (improved): `pos_weight` to counter class imbalance + early stopping + threshold tuning, explored as both global and per-class thresholds. A probability floor ensures that only sufficiently confident signals pass, and the final `inference_config` saves the tuned thresholds for reproducibility (Figure 2.3). In effect, Approach 2 treats threshold selection as part of the modeling pipeline rather than an afterthought, which better aligns the decision rule with the empirical score distributions.

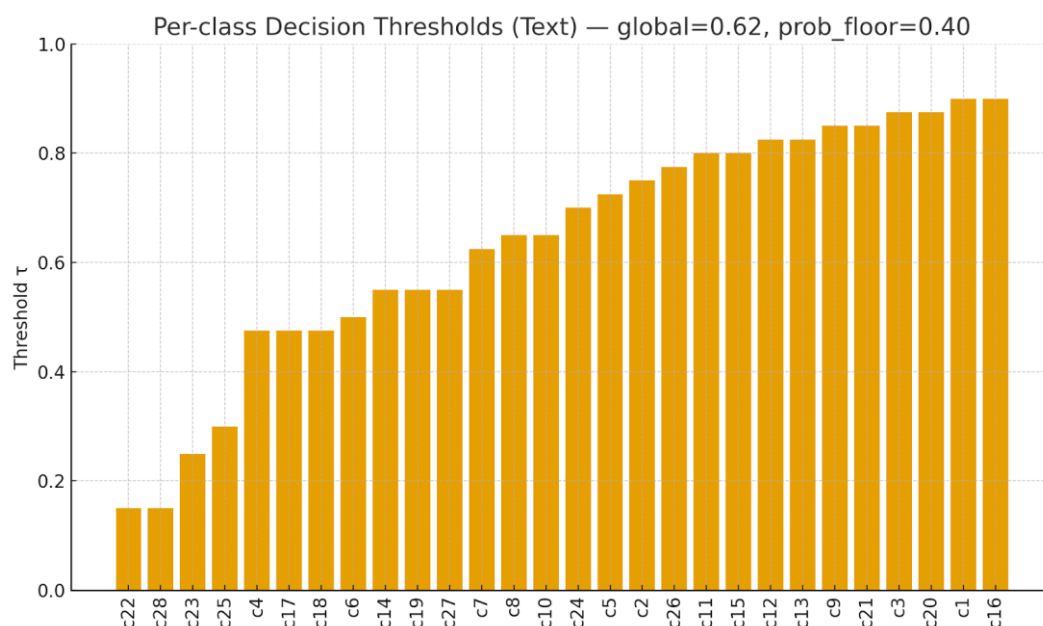


Figure 2.3 The selected per-class thresholds (τ) tuned on validation, alongside the global τ and probability floor.

Figure 2.3 shows the selected per-class thresholds (τ) tuned on validation, alongside the global τ and probability floor. Tail emotions require lower τ to preserve recall; frequent labels tolerate higher τ to improve precision (threshold data source: `inference_config.json`). Read this figure as the link between calibrated probabilities and task-specific operating points—i.e., how the same model can be steered toward different precision/recall trade-offs depending on label rarity.

The combination targets three known issues in multi-label emotion NLP: (1) imbalance across many tail emotions; (2) calibration of probabilities across classes; and (3) threshold sensitivity—where small threshold shifts can change macro-F1 notably on rare labels. GoEmotions’ own baseline results and analysis motivate such improvements. These choices also pave the way for a coherent cross-modal evaluation later, since the text side’s calibrated decision rules can be compared more fairly to single-label audio predictions under matched, task-appropriate metrics.

2.7.2 Audio as single-label

Each input is assigned one dominant emotion. SpeechBrain’s recipe-driven design makes it easy to “conceive, compare and share novel speech processing pipelines” (Ravanelli et al., 2021, p. 1), matching common SER practice. The output layer uses softmax over the 8 categories; training minimizes negative log-likelihood. This matches the annotation protocol of acted-speech datasets and keeps the decision rule straightforward and comparable to common SER practice. The SpeechBrain stack cleanly supports this formulation and yields transparent experiment folders (hyperparams, logs).

2.8 Cross-modal alignment

To compare modalities, the thesis defines a many-to-one mapping from text labels to audio categories:

happy ← joy, amusement, approval, gratitude, optimism

sad ← sadness, disappointment, grief, remorse

angry ← anger, annoyance, disapproval

fearful ← fear, nervousness (anxiety)

surprised ← surprise

disgust ← disgust

neutral/calm ← neutral (and low-arousal, matter-of-fact lines)

GoEmotions itself notes that these relations allow grouping into higher-level categories “if desired for a downstream task” (Demszky et al., 2020, p. 4041). This alignment layer is conceptual (no retraining), but it enables apples-to-apples comparisons at a rougher granularity, and it supports confusion analysis (angry ↔ disgust or calm ↔ neutral). The cost is information loss—fine shades like gratitude vs. joy collapse into

happy—so rough results are always reported alongside fine-grained text metrics. This evaluation stance mirrors MOSEI-style practice—establish strong unimodal results, then analyze how and when modalities complement each other under a shared comparison space. (Multimodal Language Analysis in the Wild: CMU-MOSEI Dataset and Interpretable Dynamic Fusion Graph)

2.9 Reliability, validity, and bias considerations

Even with tuned thresholds and carefully chosen operating points, dataset origins still matter. The GoEmotions authors document domain and annotator biases (Reddit, demographics, filtering) (Demszky et al., 2020, p. 4042). Findings may not generalize to all communities or registers; multi-label thresholds tuned on development data help but don't eliminate shift. Recognizing these limits on the text side motivates a parallel check of assumptions on the audio side.

For audio, the RAVDESS-style setting is acted and controlled, so informal, in-the-wild trials are necessary to reveal generalization gaps. SpeechBrain emphasizes simplicity, modularity, and portability, backed by recipes and pretrained models (Ravanelli et al., 2021, p. 1). Acted speech supplies clean prosody and balance, but models trained there can degrade in casual, in-the-wild conditions (background noise, channel variability, mixed emotion). This thesis therefore pairs formal dataset tests with informal app trials to surface such gaps (later chapters). To make any gap analysis meaningful, the experimental scaffolding must be reproducible across runs and machines.

Accordingly, using BERT and SpeechBrain with saved configs, seeds, and logs supports replicability. Tooling choices (PyTorch + open recipes) promote transparent experiments across machines (GPU/CPU) and make hardware effects auditable if behavior differs. This reproducibility layer complements the evaluation design and enables fairer comparisons between modalities as the system is exercised beyond the lab.

Finally, because emotion is sensitive, the prototype stores no raw audio and hashes textual inputs during trials; results are kept local in a light-weight database, aligning with the privacy posture set out in the project plan. These safeguards ensure that methodological thoroughness is paired with responsible handling of user data, the topic revisited when presenting the app-based evaluations and error analyses in the following chapters.

2.10 Adopted conceptual framework

This thesis unifies its approach based on three points. First is modality-aware modeling: the text pipeline treats emotion recognition as a multi-label problem, using sigmoid activations with BCE, imbalance-aware training, and tuned decision thresholds; the audio pipeline treats it as single-label speech emotion recognition with softmax and negative log-likelihood, following standard SER practice. These choices respect the annotation semantics of each dataset and improve decision quality for their respective inputs.

Second, the work defines comparable evaluation spaces. For text, it reports fine-grained, per-label metrics appropriate to multi-label classification, while for audio it reports categorical metrics aligned with single-label evaluation. A mapping layer then projects both outputs into a shared, rough label space, enabling meaningful cross-modality comparison without distorting either modality's native label structure.

Finally, evaluation proceeds in dual contexts. Formal testing uses the public test splits for each dataset to establish baseline performance and replicability. Additionally, the mobile application prototype captures real-world usage, showing shift and practical failure cases.

3 RESEARCH METHODOLOGY AND APPROACH

This chapter contains information regarding the datasets and everything related to them. For instance, label spaces, preprocessing pipelines, and reproducibility practices as well as other things that used in the thesis. Throughout, design choices are modality-aware—multi-label text versus single-label audio—and grounded in standard toolchains for NLP and speech (Devlin et al., 2019; Demszky et al., 2020; Ravanelli et al., 2021).

3.1 Datasets and label spaces

The text experiments use GoEmotions, a corpus of approximately 58k Reddit comments annotated with 27 fine-grained emotions plus Neutral and allowing multi-label ground truth (Demszky et al., 2020). The official train/validation/test splits are used as provided to avoid split leakage. Figure 3.1 shows representative test rows to make the multi-label structure concrete: each comment is paired with one or more label indices rather than a single categorical tag (Fig. 3.1).

[NAME] has towed the line of the Dark Side. He wouldn't cross it by doing something like this.	3,10	eeaf7fg
Lol! But I love your last name though. XD	1,18	ee3yyz3
Translation }}} I wish I could afford it.	8	eegij2q
It's great that you're a recovering addict, that's cool. Have you ever tried DMT?	0,7	eeccgbb
I've also heard that intriguing but also kinda scary	14	edk4e66

Figure 3.1 GoEmotions test samples with multi-label annotations (label indices).

For speech, an acted RAVDESS-style subset with eight categorical emotions is used in the thesis. Splits are stratified by emotion and stored in simple CSVs with following columns ID, wav, duration, and emotion, which keeps the pipeline transparent and scriptable; a preview of the test file is shown in Fig. 3.2. Canonical label orders are serialized in small JSON files for both modalities—28 entries for text (Fig. 3.3) and eight for audio (Fig. 3.4)—and are treated as single sources of truth at training and inference time.

	A	B	C	D
1	ID	wav	duration	emotion
2	03_01_03_01_01_0	/content/drive/MyDriv	3.7370625	happy
3	03_01_08_02_02_0	/content/drive/MyDriv	3.470125	surprised
4	03_01_03_01_01_0	/content/drive/MyDriv	3.703708333	happy
5	03_01_04_02_01_0	/content/drive/MyDriv	3.403395833	sad
6	03_01_08_02_01_0	/content/drive/MyDriv	4.104104167	surprised
7	03_01_04_02_01_0	/content/drive/MyDriv	3.336666667	sad
8	03_01_06_02_02_0	/content/drive/MyDriv	3.536875	fearful
9	03_01_04_01_01_0	/content/drive/MyDriv	3.570229167	sad

Figure 3.2 RAVDESS test split preview (ID, wav, duration, emotion).

```
[
  "admiration","amusement","anger","annoyance","approval",
  "caring","confusion","curiosity","desire","disappointment",
  "disapproval","disgust","embarrassment","excitement","fear",
  "gratitude","grief","joy","love","nervousness",
  "optimism","pride","realization","relief","remorse",
  "sadness","surprise","neutral"
]
```

Figure 3.3 Canonical label order for text (28 labels).

```
[
  "angry",
  "calm",
  "disgust",
  "fearful",
  "happy",
  "neutral",
  "sad",
  "surprised"
]
```

Figure 3.4 Canonical label order for audio (8 labels).

To enable a rough cross-modal comparison later in the results, the fine-grained text labels are mapped to the eight audio categories (e.g., happy ← [joy, amusement, approval, gratitude, optimism]). This mapping is used only for analysis and never for training, preserving each modality's native label semantics.

3.2 Preprocessing pipelines

On the text side, inputs are tokenized with bert-base-uncased using truncation to 128 tokens and dynamic padding, a standard configuration that balances context coverage with batch efficiency (Fig. 3.5). Because the task is multi-label, target lists are converted into multi-hot vectors (float32) so that each class can be optimized independently under a sigmoid/BCE objective. A light custom collator ensures labels remain float32 after padding and batching.

```
# Tokenizer
tokenizer = globals().get("tokenizer", None) or AutoTokenizer.from_pretrained(MODEL_NAME)

# Preprocess
def to_multihot(labels, K, dtype=np.float32):
    v = np.zeros(K, dtype=dtype)
    for i in labels: v[i] = 1.0
    return v

def preprocess(batch):
    enc = tokenizer(batch["text"], truncation=True, max_length=128)
    enc["labels"] = [to_multihot(lbls, num_labels) for lbls in batch["labels"]]
    return enc

encoded = globals().get("encoded", None) or ds.map(
    preprocess, batched=True, remove_columns=ds["train"].column_names
)

# Collator to force float32 labels
class DataCollatorForMultiLabel(DataCollatorWithPadding):
    def __call__(self, features):
        batch = super().__call__(features)
        batch["labels"] = torch.tensor([f["labels"] for f in features], dtype=torch.float32)
        return batch

data_collator = DataCollatorForMultiLabel(tokenizer=tokenizer)
```

Figure 3.5 Text tokenization and batching configuration (bert-base-uncased, max length 128, dynamic padding).

For audio, raw WAV files are loaded, converted to mono, and, when necessary, resampled to 16 kHz to match the feature extractor. The acoustic front-end computes log-Mel spectrograms with $n_{\text{fft}}=1024$, $\text{hop_length}=256$, and $n_{\text{mels}}=80$ (Fig. 3.6). Variable-length utterances are batch-padded along the time axis and their lengths carried forward for bookkeeping. These choices align with common SER practice and with off-the-shelf recipes in SpeechBrain (Ravanelli et al., 2021).

```

# Features (log-Mel)
mel_spec = torchaudio.transforms.MelSpectrogram(
    sample_rate=SAMPLE_RATE, n_fft=1024, hop_length=256, n_mels=80
)
ampl_to_db = torchaudio.transforms.AmplitudeToDB()

def compute_features(sig_batch):
    feats_list = []
    for sig in sig_batch:
        S = mel_spec(sig) # [n_mels, T]
        S_db = ampl_to_db(S).transpose(0, 1) # [T, n_mels]
        feats_list.append(S_db)
    lens = torch.tensor([f.size(0) for f in feats_list], dtype=torch.long)
    max_len = int(lens.max().item())
    feat_dim = feats_list[0].size(1)
    padded = torch.zeros(len(feats_list), max_len, feat_dim)
    for i, f in enumerate(feats_list):
        T = f.size(0)
        padded[i, :T, :feat_dim] = f
    return padded, lens

```

Figure 3.6 Audio feature extractor (log-Mel: $n_fft=1024$, $hop=256$, $n_mels=80$) and batch padding.

3.3 Models and objectives

For text, the classifier is a BERT-base encoder (Devlin et al., 2019) with a 28-unit linear head and sigmoid activations, treating each emotion as an independent Bernoulli target. This formulation matches GoEmotions’ multi-label annotation scheme (Demszky et al., 2020) and yields probabilities calibrated per class. Two training strategies are considered. The baseline uses uniform BCEWithLogits and a fixed 0.50 decision threshold at inference. The improved strategy adds class-imbalance compensation via `pos_weight` derived from training prevalence, an early-stopping criterion on development performance, and a threshold-tuning step that searches both a global cutoff and per-class cutoffs on the validation set. The small snippet in Fig. 3.7 illustrates the weighted-loss hook inside a light Trainer subclass used to pass `pos_weight` to BCEWithLogitsLoss.

```

# Class weights for BCEWithLogits (pos_weight)
Y = np.array(encoded["train"]["labels"], dtype=np.float32)
pos = Y.sum(axis=0)
neg = Y.shape[0] - pos
pos_weight = torch.tensor((neg + 1e-3) / (pos + 1e-3), dtype=torch.float32)
pos_weight = torch.clamp(pos_weight, max=10.0) # cap extremes for stability

# Weighted Trainer (uses pos_weight)
class WeightedTrainer(Trainer):
    def __init__(self, pos_weight=None, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.pos_weight = pos_weight

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits
        loss_fct = torch.nn.BCEWithLogitsLoss(pos_weight=self.pos_weight.to(logits.device))
        loss = loss_fct(logits, labels)
        return (loss, outputs) if return_outputs else loss

```

Figure 3.7 Weighted BCE implementation for multi-label text: pos_weight computation and custom trainer hook.

For speech, the model is a compact CRDNN: two convolutional blocks (5×5 then 3×3 with batch-norm, ReLU, and pooling) feed a BiGRU(128), followed by temporal mean-pooling and a small MLP classifier. The output is a single softmax over the eight emotions, optimized with negative log-likelihood. This single-label setup mirrors the acted-speech annotations and follows a standard SER recipe structure (Ravanelli et al., 2021). Figure 3.8 shows the dataset object graph and collation used to pair signals with label indices and to construct the training, validation, and test dataloaders.

```

# Pipelines (audio and labels)
def audio_pipeline(wav_path):
    sig, sr = torchaudio.load(wav_path)
    if sig.shape[0] > 1:
        sig = sig.mean(dim=0, keepdim=True) # mono
    if sr != SAMPLE_RATE:
        sig = torchaudio.transforms.Resample(orig_freq=sr, new_freq=SAMPLE_RATE)(sig)
    return sig.squeeze(0)

def label_pipeline(idx):
    return torch.tensor(idx, dtype=torch.long)

for split in datasets:
    datasets[split].add_dynamic_item(audio_pipeline, takes="wav", provides="sig")
    datasets[split].add_dynamic_item(label_pipeline, takes="emotion", provides="label")
    datasets[split].set_output_keys(["id", "sig", "label"])

# Dataloaders
BATCH_SIZE = 16

def collate_fn(batch):
    sigs = [b["sig"] for b in batch]
    labels = torch.tensor([int(b["label"].item()) for b in batch], dtype=torch.long)
    return ("sig": sigs, "label": labels)

train_dataloader = DataLoader(datasets["train"], batch_size=BATCH_SIZE, shuffle=True, collate_fn=collate_fn)
valid_dataloader = DataLoader(datasets["valid"], batch_size=BATCH_SIZE, shuffle=False, collate_fn=collate_fn)
test_dataloader = DataLoader(datasets["test"], batch_size=BATCH_SIZE, shuffle=False, collate_fn=collate_fn)

```

Figure 3.8 Audio dataset and dataloader setup (signal/label pipelines; batch collation).

3.4 Training configuration and schedules

Text fine-tuning adheres to widely used defaults: Adam with decoupled weight decay with learning rate $2e-5$, linear schedule with 6 % warm-up, weight decay 0.01, batch size 16/32 (train/eval), and up to 5 epochs with early stopping (the baseline is capped at 3). Mixed precision is enabled on CUDA hosts. Micro-F1 on the validation split selects checkpoints, aligning the model objective with the downstream evaluation. For audio, training runs 25 epochs with Adam ($1e-3$) and batch size 16, monitoring validation accuracy to pick the best checkpoint. All experiments fix a seed and, for audio, enable deterministic cuDNN kernels; although this slightly reduces throughput, it stabilizes results across hosts.

3.5 Inference and exported artifacts

Because multi-label decisions are threshold-sensitive, the text system separates probability estimation from decision calibration. After training, logits are converted to probabilities via sigmoid (Fig. 3.9) and a small grid search identifies either a single global threshold or per-class thresholds that optimize validation performance. The selected configuration—together with an optional probability floor that suppresses very low-confidence activations—is serialized to `inference_config.json` and loaded at runtime. Figure 3.10, produced directly from that file, visualizes the per-class thresholds and annotates the chosen global τ and probability floor. Calibration happens strictly on the validation split; no test peeking occurs. At inference time, the backend returns both the binary label set and per-label probabilities (with the option to expose top-k for inspection).


```
sigmoid = lambda x: 1.0 / (1.0 + np.exp(-x))
val_probs = sigmoid(val_logits)
test_probs = sigmoid(test_logits)
```

Figure 3.9 Post-training probability computation via sigmoid for validation/test logits.

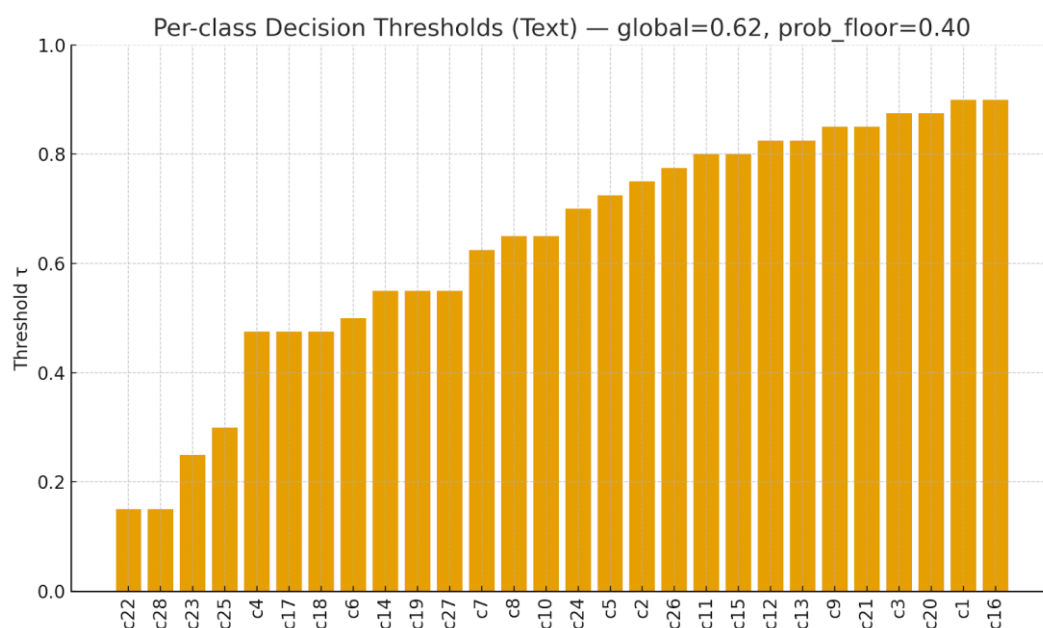


Figure 3.10 Per-class decision thresholds for text (global τ and probability floor shown).

For audio, inference is a straightforward argmax over softmax outputs, with probabilities reported for interpretability. Confusion matrices (counts and row-normalized) are generated post-hoc to analyze error modes and are referenced later in the Results chapter.

3.6 Evaluation protocol and metrics

Two complementary evaluation contexts are used. Formal evaluation reports metrics on the predefined test splits: for text, micro/macro precision, recall, and F1, subset accuracy, and a per-class table; for audio, accuracy, macro precision/recall/F1, and the two confusion matrices. These choices reflect each modality’s label semantics and are consistent with prior work (Demszky et al., 2020; Ravanelli et al., 2021). Informal

evaluation is conducted through the mobile prototype: users submit short texts or speech, receive predictions, and provide a star rating with an optional comment. These interactions are logged locally in a small SQLite database (SHA-256 hashed finger-prints of audio and text; no raw text and audio data is stored) and later summarized to inspect runtime, common confusions, and confidence patterns.

3.7 Reproducibility and implementation notes

All experiments record the software environment (framework versions, CUDA/cuDNN), random seeds, and the exact label orders consumed by models at train and test time (Fig. 3.3–3.4). Text runs set framework seeds; audio runs additionally force deterministic cuDNN to prevent kernel drift. To guard against instability from extreme imbalance, `pos_weight` values derived from class prevalence are clamped to an upper bound for numerical stability (see Fig. 3.7). Because threshold calibration depends on class priors, the text system treats thresholds as dataset-specific artifacts; if priors change materially, thresholds should be refreshed on a new development split. Finally, variable-length audio batches are padded to the longest sequence within a batch; future work could incorporate packed sequences or attention masks, but the present recipe is intentionally minimal and transparent.

4 IMPLEMENTATION AND SYSTEM DESIGN

This chapter explains how the design turned was into a working system. It describes the React Native/Expo client and FastAPI backend, the two model adapters (text and audio), and the end-to-end request loop from mobile input to prediction and feedback. It covers preprocessing (tokenization vs. log-Mel features), inference (sigmoid/BCE with calibrated thresholds vs. softmax/NLL), on-device SQLite feedback storage, and a local-only deployment workflow (WSL + portproxy). The chapter's goal is operational clarity: minimal endpoints, deterministic inference, auditable artifacts, and a UX that makes collecting informal evidence as straightforward as running formal tests.

4.1 Design goals and constraints

The prototype was built to make two modeling lines—multi-label text and single-label speech—usable from a phone while keeping the research surface small and reproducible. All choices follow three constraints from the project plan: a privacy-first, a fully local setup that avoids paid deployment, and transparent artifacts that an examiner can re-run. Toolkits and datasets then dictate the rest: a BERT pathway fine-tuned on GoEmotions for text, and a compact SpeechBrain CRDNN trained on an acted-speech corpus for audio (Devlin et al., 2019; Demszky et al., 2020; Ravanelli et al., 2021).

4.2 System architecture (high-level)

At a high level the system consists of a React Native/Expo client that talks to a FastAPI server hosting two model adapters, one for text and one for audio. The adapters load their model weights and preprocessing configurations, apply the same tokenization or acoustic features used during finte-tuning, and return predictions to the client. A small SQLite database on the device preserves only structured feedback and

metadata. Figure 4.1 shows the end-to-end arrangement and the direction of data flow between client, server, models, and the local store (Fig. 4.1).

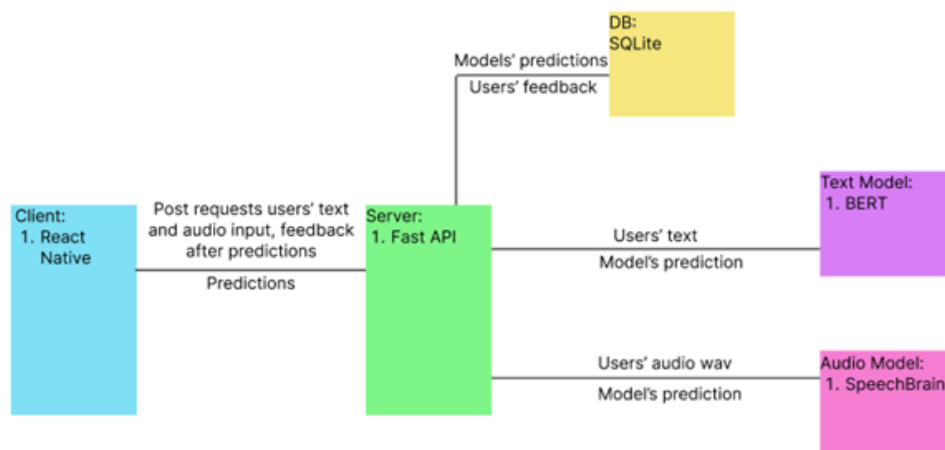


Figure 4.1 System diagram.

4.3 Data flow from phone to model and back

A typical interaction begins when the user types a message or records a short spoken sentence in the app. The client posts the request to the corresponding FastAPI route. On the text path, the server tokenizes the message with the same bert-base-uncased settings used in fine-tuning, performs inference, converts logits to probabilities, and applies the calibrated thresholds saved in `inference_config.json` to decide which labels fire. On the audio path, the server accepts the uploaded clip, converts it to WAV with `ffmpeg` when necessary, resamples to 16 kHz, computes log-Mel spectrograms, and feeds them to the CRDNN to obtain a softmax distribution and a top label. The response returns predicted labels and probabilities together with a unique `prediction_id`. Immediately after viewing a result, the user can submit a star rating and an optional comment; the client stores this feedback locally, along with the `prediction_id`, for later analysis. Throughout the loop, raw audio is never stored as well as raw text, and both have finger-print digital hashing, which serves

as meta-data (e.g, to compare if messages are the same) (SHA-256) before persistence.

4.4 Backend implementation

The server is intentionally small and focuses on deterministic inference. The text adapter loads BERT and its tokenizer, mirrors the fine-tuning truncation and padding policy, and applies the saved global or per-class thresholds (with an optional probability floor) to produce multi-label decisions. The audio adapter loads the CRDNN, enforces a mono 16 kHz input, computes log-Mel features, and emits a single arg-max class with the full probability vector for interpretability. Both adapters include the model name and version in the response for auditability. Example responses captured in Postman are shown for /api/text, /api/audio, and /api/feedback (Fig. 4.2–4.3), and a simple /api/healthz route reports liveness and which models are loaded.

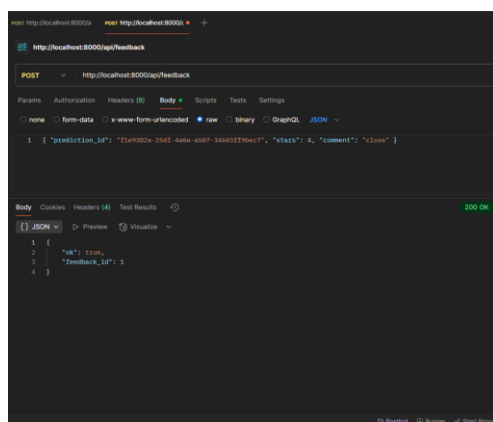
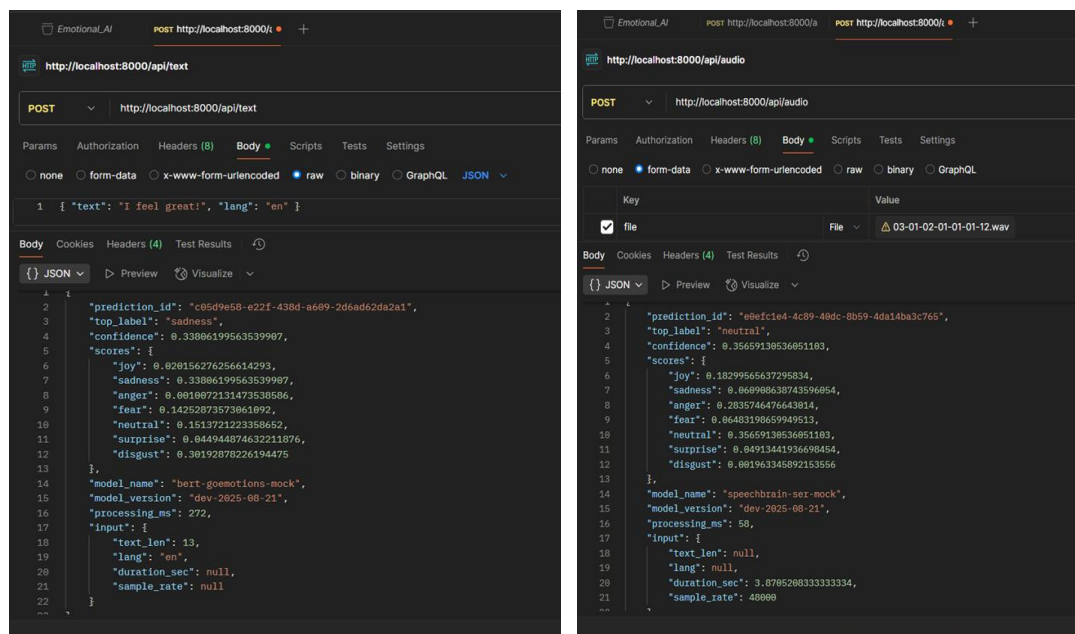


Figure 4.2 Feedback API endpoint example.



(a) Text prediction

(b) Audio prediction

Figure 4.3 Two examples of API endpoints: (a) text; (b) audio

4.5 API surface and validation

The API surface is deliberately thin: three POST routes handle text predictions, audio predictions, and feedback; two GET routes serve analytics and health checks. Server-side validation ensures required fields and MIME types are present; requests that fail validation yield clear 4xx messages, and unexpected exceptions are sanitized as 5xx responses. To keep the backend simple there is no throttle logic or JWT authentication; instead, the client applies practical limits—512 characters for text and 30 seconds for audio—so payload sizes remain bounded during trials.

4.6 Mobile client

The app presents three simple surfaces: a home screen, a text screen, and an audio screen. The home screen routes to the two modalities and an analytics view (Fig. 4.4 (a)). On the audio screen, the recorder is permission-gated and capped at 30 s; after sending the clip the user

sees the predicted label with its distribution, then submits a rating (Fig. 4.4 (b)). On the text screen, the multi-label probabilities are displayed and the same feedback flow follows (Fig. 4.4 (c); (d)). The UX is intentionally quiet: loading states disable repeated taps, errors from the server are surfaced clearly, and probabilities are presented as percentages. A tiny API client centralizes the base URL and JSON/multipart helpers so the UI code remains clean.

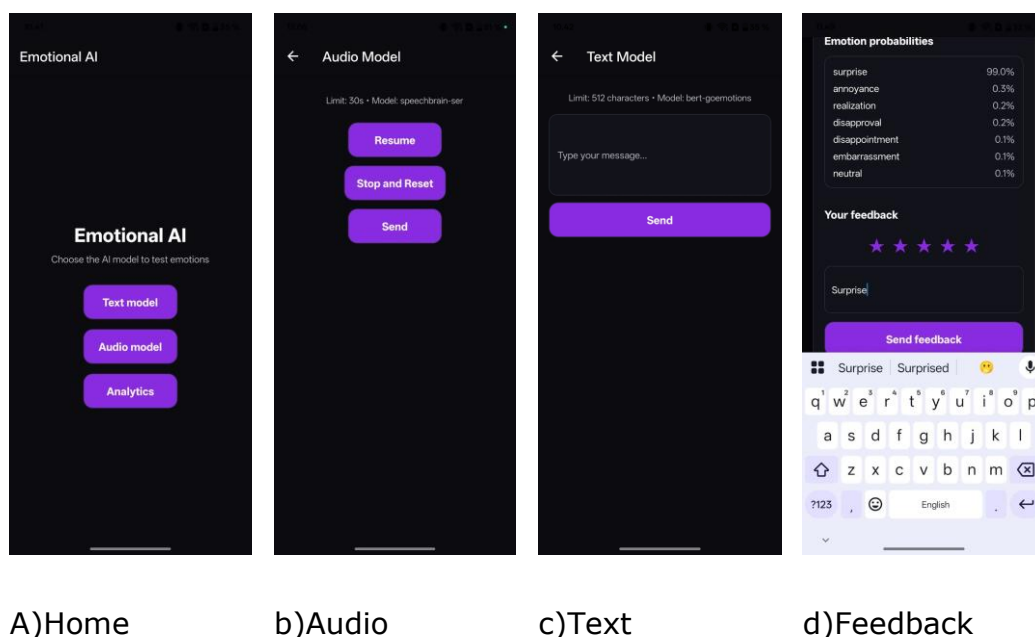


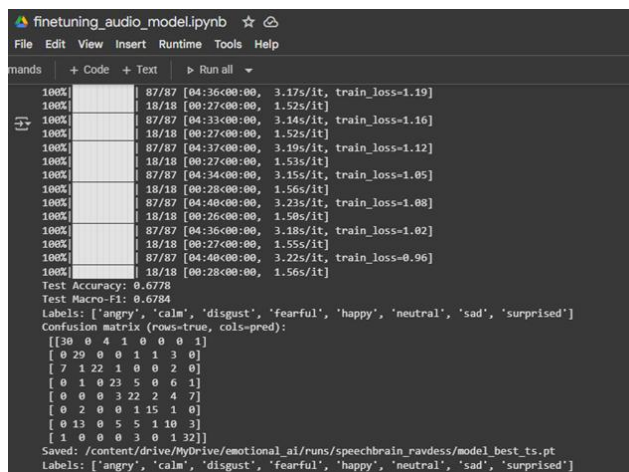
Figure 4.4 Mobile application screens: a) Home; b) Audio; c) Text; d) Feedback

4.7 Persistence, privacy, and analytics

Only two SQLite tables are maintained on-device. The predictions table carries a unique identifier, timestamps, modality, basic input metadata (text length or audio duration and sample rate), the model name and version, the top label and its confidence, processing time, the full scores dictionary, and hash for text and audio. The feedback table references a prediction and stores the star rating, optional comment, submission time, and basic client info. This design enables analytics such as accuracy-by-stars or distribution by label without ever storing raw messages or audio.

4.8 Runtime environment and deployment

All experiments and local runs were executed without paid hosting. The backend runs in WSL on a laptop; the app runs on a OnePlus Nord CE 4 Lite. During development the phone connects to the laptop's private hotspot, and Windows portproxy forwards the public laptop address to the FastAPI process running in WSL. Firewall rules restrict access to the local subnet. Training for the text model was done in the standard environment described earlier; the audio model was fine-tuned first on CPU and then on a T4 GPU in Google Colab as a small experiment to contrast performance; representative logs for both runs are included in Fig. 4.5–4.6.

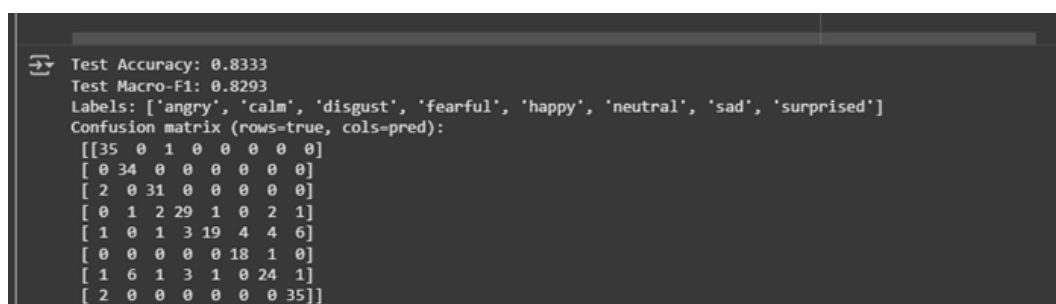


```

finetuning_audio_model.ipynb
File Edit View Insert Runtime Tools Help
+ Code + Text ▶ Run all
100% [87/87 [04:36<00:00, 3.17s/it, train_loss=1.19]
100% [18/18 [00:27<00:00, 1.52s/it]
100% [87/87 [04:33<00:00, 3.14s/it, train_loss=1.16]
100% [18/18 [00:27<00:00, 1.52s/it]
100% [87/87 [04:32<00:00, 3.19s/it, train_loss=1.12]
100% [18/18 [00:27<00:00, 1.53s/it]
100% [87/87 [04:34<00:00, 3.15s/it, train_loss=1.05]
100% [18/18 [00:28<00:00, 1.56s/it]
100% [87/87 [04:40<00:00, 3.23s/it, train_loss=1.08]
100% [18/18 [00:26<00:00, 1.50s/it]
100% [87/87 [04:36<00:00, 3.18s/it, train_loss=1.02]
100% [18/18 [00:27<00:00, 1.55s/it]
100% [87/87 [04:40<00:00, 3.22s/it, train_loss=0.96]
100% [18/18 [00:28<00:00, 1.56s/it]
Test Accuracy: 0.6778
Test Macro-F1: 0.6784
Labels: ['angry', 'calm', 'disgust', 'fearful', 'happy', 'neutral', 'sad', 'surprised']
Confusion matrix (rows=true, cols=pred):
[[38 0 4 1 0 0 0 1]
 [0 29 0 0 1 1 3 0]
 [7 1 22 1 0 0 2 0]
 [0 1 0 23 5 0 6 1]
 [0 0 0 3 22 2 4 7]
 [0 2 0 0 1 15 1 0]
 [0 13 0 5 5 1 10 3]
 [1 0 0 0 3 0 1 13]]
Saved: /content/drive/MyDrive/emotional_ai/runs/speechbrain_ravdess/model_best.ts.pt
Labels: ['angry', 'calm', 'disgust', 'fearful', 'happy', 'neutral', 'sad', 'surprised']

```

Figure 4.5 Audio fine-tuning on CPU.



```

Test Accuracy: 0.8333
Test Macro-F1: 0.8293
Labels: ['angry', 'calm', 'disgust', 'fearful', 'happy', 'neutral', 'sad', 'surprised']
Confusion matrix (rows=true, cols=pred):
[[35 0 1 0 0 0 0 0]
 [0 34 0 0 0 0 0 0]
 [2 0 31 0 0 0 0 0]
 [0 1 2 29 1 0 2 1]
 [1 0 1 3 19 4 4 6]
 [0 0 0 0 0 18 1 0]
 [1 6 1 3 1 0 24 1]
 [2 0 0 0 0 0 0 35]]

```

Figure 4.6 Audio fine-tuning on T4 GPU.

4.9 System Efficiency and Constraints

The end-to-end processing time—from the moment the mobile app sends a request to the display of the predictions—averages approximately 458 ms for text and 500 ms for audio on the local setup described earlier. These results demonstrate the efficiency of using small, well-calibrated models and a fully local deployment strategy. The latency remains low enough to support smooth user interaction and near real-time feedback, highlighting the practicality of lightweight architectures in emotion recognition tasks.

To enable cross-modal comparison without altering the independent training pipelines, the prototype applies a conceptual label mapping. The mapping aggregates the text system’s fine-grained labels into the eight broader emotion categories used by the audio classifier. For example, happy unites labels such as joy, amusement, approval, gratitude, and optimism, while neutral aligns with neutral/calm. The mapping is used strictly for presentation and analysis; all training, validation, and evaluation remain modality-specific to preserve the integrity of the results.

Several limitations and trade-offs are inherent to this design. The prototype purposely avoids features that would complicate experimentation, such as authentication mechanisms, endpoint throttling, or cloud-based deployment. On the audio side, dataset splits stratified by emotion rather than speaker disjointness may increase apparent generalization performance; a production-level speech emotion recognition system would require stricter partitioning and more robust data augmentation. On the text side, calibrated decision thresholds are influenced by dataset priors and may need retuning when class balance or domain characteristics shift. Despite these constraints, the system achieves its research goals: it operationalizes the modeling assumptions introduced in Chapters 2 and 3—sigmoid/BCE with per-class thresholds for text and softmax/NLL for speech—within a privacy-preserving, fully local, and reproducible prototype.

5 RESULTS

This chapter reports formal results on each dataset’s held-out test split, informal results gathered through the mobile prototype, a rough cross-modal comparison using the alignment described earlier, and brief error analyses and validity notes. Text uses the GoEmotions split with a BERT-base head and multi-label decision thresholds tuned on development data; audio uses an acted-speech split with a compact CRDNN trained and evaluated as a single-label classifier.

5.1 Evaluation setup (recap)

For text, the final reporting configuration utilizes per-class thresholds saved to `inference_config.json`; overall test metrics come from `metrics_overall.json` and per-class metrics from the exported CSV. For audio, overall test metrics come from `metrics_overall.json`, and error patterns are visualized with count and row-normalized confusion matrices. Formal tests always use the official test splits. Informal results are collected via the app with on-device SQLite, which is different from the project plan, which stated that raw text would be stored.

5.2 Text modality — formal test results

On the held-out test set, the text model attains $\text{micro-F1} = 0.566$, $\text{macro-F1} = 0.52$, $\text{micro-precision} = 0.53$, $\text{micro-recall} = 0.607$, and $\text{subset accuracy} = 0.409$ with 28 labels over $N = 5,427$ samples; the decision rule applies per-class thresholds previously tuned on validation data. These values are read from `metrics_overall.json` artifact, which records the chosen “strategy”: “per-class.”

Per-class behavior reflects the usual head–tail split on GoEmotions: strong categories include gratitude, amusement, and love, whereas diffuse or context-heavy labels such as realization, disappointment, annoyance, and confusion remain comparatively weak—even after imbalance handling and threshold tuning (see per-class metrics CSV). The

threshold calibration step proved essential: moving from a fixed 0.50 cutoff to per-class cutoffs improved precision/recall trade-offs, especially on rare emotions (see Fig. 5.3 below for the threshold profile).

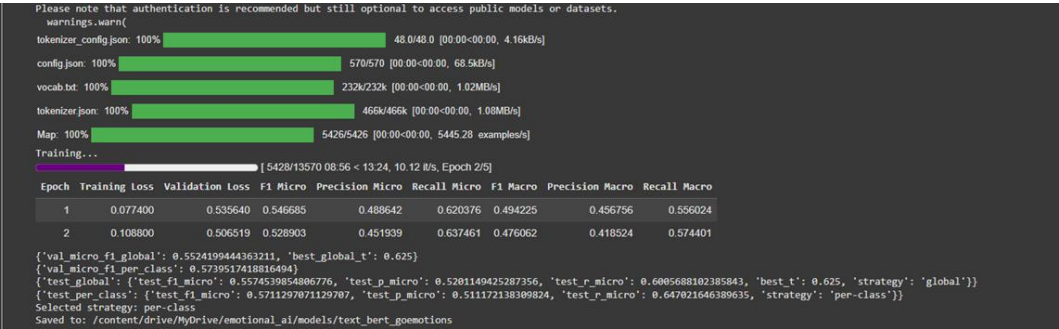


Figure 5.1 Text training improved run.

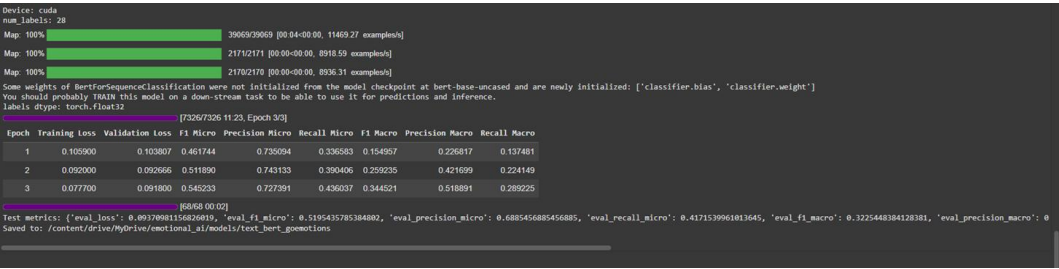


Figure 5.2 Text baseline run.

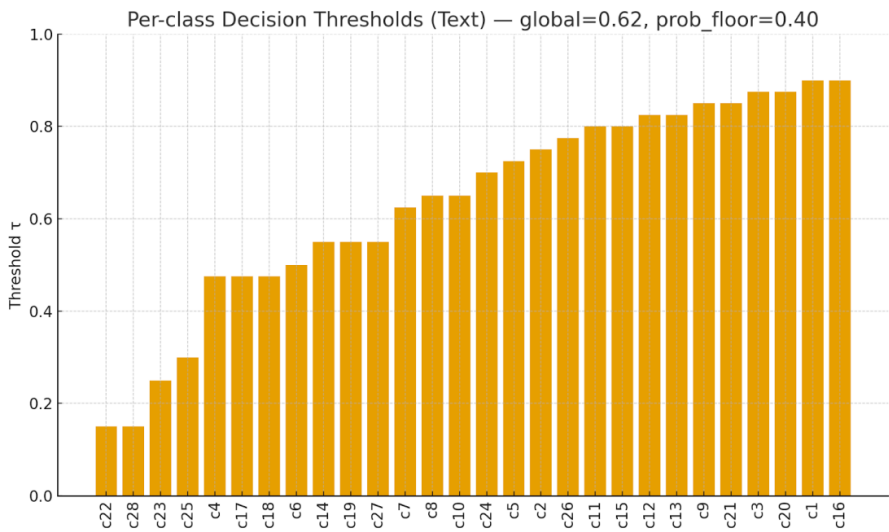
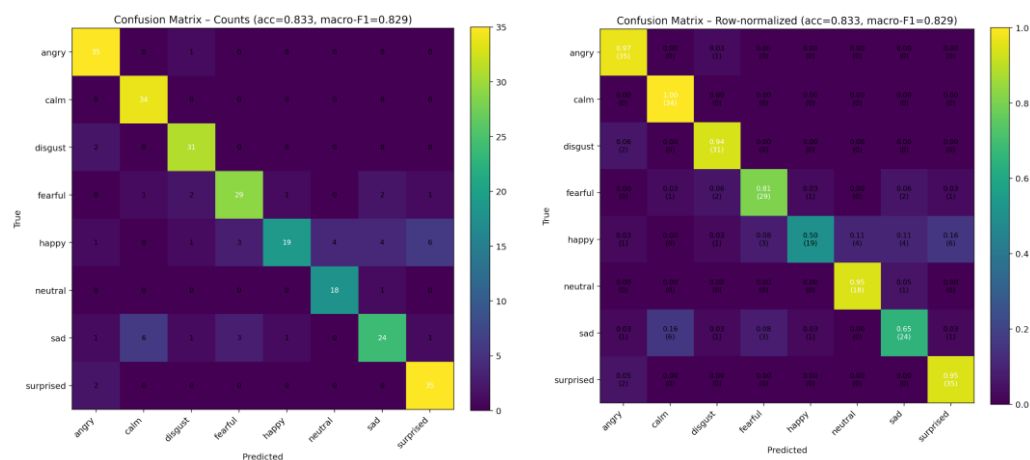


Figure 5.3 Per-class thresholds.

5.3 Audio modality — formal test results

The audio model reaches accuracy = 0.833 and macro-F1 = 0.829 on $N = 270$ inputs across 8 classes (artifact metrics_overall.json). The count and row-normalized confusion matrices show a consistent pattern: angry, calm, disgust, and surprised are robust; happy under-recalls (≈ 0.50 recall) and sad under-recalls (≈ 0.65 recall), with leakage into neutral/calm—a common prosody overlap for low-to-mid arousal speech.



a)Confusion matrix (counts)

b)Confusion matrix row-normalized

Figure 5.4 Confusion matrices.

5.4 Cross-modal comparison (rough mapping)

For analysis only, the fine-grained text labels are aggregated into the eight audio categories (e.g., happy $\leftarrow$ [joy, amusement, approval, gratitude, optimism]; neutral/calm $\leftarrow$ [neutral]). This alignment was not used for training; it serves to discuss tendencies at a shared granularity. Under this lens, text performs reliably for amusement/gratitude/joy but struggles with approval (lexically subtle). Audio is comparatively weak on happy and sometimes confuses sad with neutral/calm, which aligns with the low-energy prosodic overlap visible in the row-normalized matrix. Angry vs. disgust mix occasionally in audio due to shared sharp

spectral/temporal cues, whereas text separates disgust better when explicit words are present. Because the cross-modal comparison in the prototype was theoretical (visual inspection of app analytics rather than a fused evaluation), the thesis report these as qualitative observations rather than a numeric delta scoreboard.

5.5 Informal prototype trials (in-app)

The app analytics confirm the formal findings while detecting domain shift. On spontaneous recordings (varying mic distance, background noise, casual speaking style), the audio pathway's accuracy by user feedback sits around the high-60% range (see Fig. 5.5 App analytics (a)) (e.g., 68.6% with moderate criteria), whereas the text pathway reaches around 90% under the same informal messages on the same period (see Fig. 5.5 App analytics (b)). These panels also show average confidence and processing time, with audio clips averaging ≈ 500 ms end-to-end and text around ≈ 458 –500 ms. The breakdown per emotion highlights where app usage clustered (e.g., many calm/neutral entries).

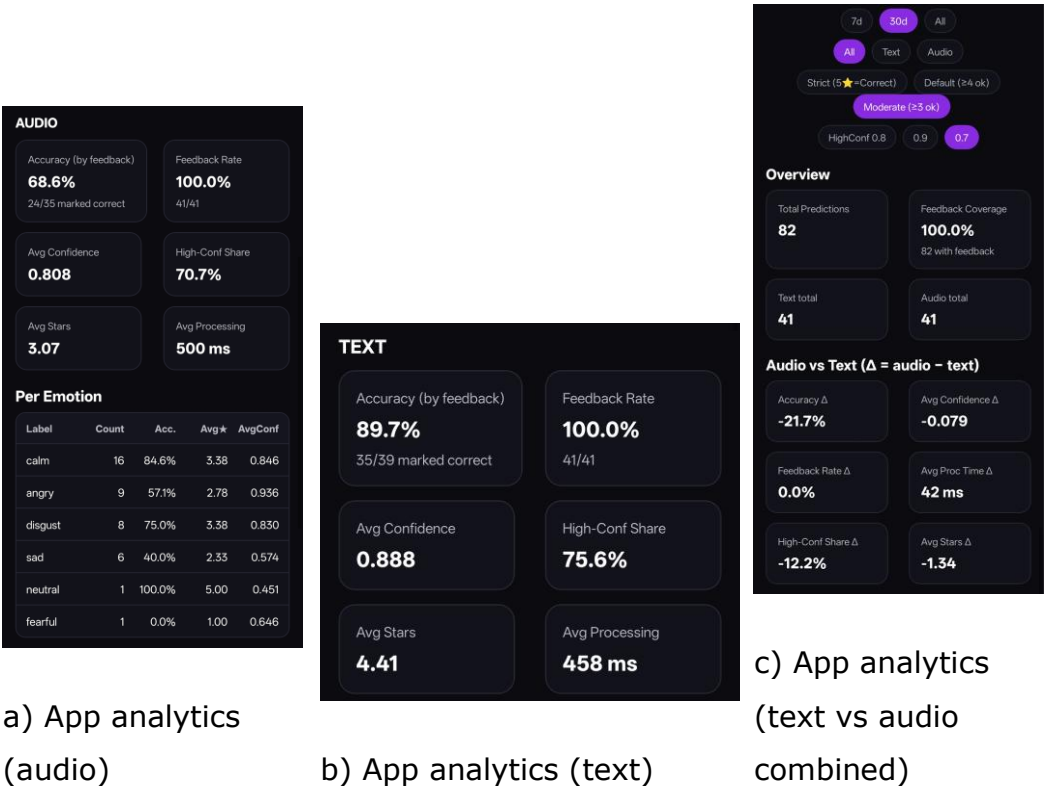


Figure 5.5 App analytics: a) audio; b) text; c) text vs audio.

5.6 Sensitivity

Two short studies illustrate sensitivity. First, thresholding on text: the improved run—pos_weight + early stopping + per-class thresholding—was adopted for test reporting and outperformed the fixed-threshold baseline (compare Figs. 5.1 and 5.2). Second, hardware on audio: fine-tuning on a T4 GPU versus CPU produced different test outcomes in exploratory runs (see Fig. 5.6 and Fig. 5.7), underscoring the need to re-validate when moving across hardware environments or kernels.

```

finetuning_audio_model.ipynb
File Edit View Insert Runtime Tools Help

100% [Progress Bar] 87/87 [04:36<00:00, 3.17s/it, train_loss-1.19]
100% [Progress Bar] 18/18 [00:27<00:00, 1.52s/it]
100% [Progress Bar] 87/87 [04:33<00:00, 3.14s/it, train_loss-1.16]
100% [Progress Bar] 18/18 [00:27<00:00, 1.52s/it]
100% [Progress Bar] 87/87 [04:37<00:00, 3.10s/it, train_loss-1.12]
100% [Progress Bar] 18/18 [00:27<00:00, 1.53s/it]
100% [Progress Bar] 87/87 [04:34<00:00, 3.15s/it, train_loss-1.05]
100% [Progress Bar] 18/18 [00:28<00:00, 1.56s/it]
100% [Progress Bar] 87/87 [04:40<00:00, 3.23s/it, train_loss-1.08]
100% [Progress Bar] 18/18 [00:26<00:00, 1.50s/it]
100% [Progress Bar] 87/87 [04:36<00:00, 3.18s/it, train_loss-1.02]
100% [Progress Bar] 18/18 [00:27<00:00, 1.55s/it]
100% [Progress Bar] 87/87 [04:40<00:00, 3.22s/it, train_loss-0.96]
100% [Progress Bar] 18/18 [00:28<00:00, 1.56s/it]
Test Accuracy: 0.6778
Test Macro-F1: 0.6784
Labels: ['angry', 'calm', 'disgust', 'fearful', 'happy', 'neutral', 'sad', 'surprised']
Confusion matrix (rows=true, cols=pred):
[[30 0 4 1 0 0 0 1]
 [0 29 0 0 1 1 3 0]
 [7 1 22 1 0 0 2 0]
 [0 1 0 23 5 0 6 1]
 [0 0 0 3 22 2 4 7]
 [0 2 0 0 1 15 1 0]
 [0 13 0 5 5 1 10 3]
 [1 0 0 0 3 0 1 32]]
Saved: /content/drive/MyDrive/emotional_ai/runs/speechbrain_ravdess/model_best_ts.pt
Labels: ['angry', 'calm', 'disgust', 'fearful', 'happy', 'neutral', 'sad', 'surprised']

```

Figure 5.6 Audio training/testing on CPU.

```

Test Accuracy: 0.8333
Test Macro-F1: 0.8293
Labels: ['angry', 'calm', 'disgust', 'fearful', 'happy', 'neutral', 'sad', 'surprised']
Confusion matrix (rows=true, cols=pred):
[[35 0 1 0 0 0 0 0]
 [0 34 0 0 0 0 0 0]
 [2 0 31 0 0 0 0 0]
 [0 1 2 29 1 0 2 1]
 [1 0 1 3 19 4 4 6]
 [0 0 0 0 0 18 1 0]
 [1 6 1 3 1 0 24 1]
 [2 0 0 0 0 0 0 35]]

```

Figure 5.7 Audio training/testing on T4 GPU.

5.7 Error analysis and threats to validity

On text, the most error-prone labels are tail or diffuse categories (e.g., realization, disappointment), where semantic cues are subtle and context-dependent. Per-class thresholds help balance precision/recall but cannot fully compensate for label sparsity. On audio, happy and sad under-recall, reflecting the acoustic proximity to nearby classes (cheerful vs. neutral, low-energy sadness vs. calm). The formal audio test set is acted; informal results reveal the expected drop in the wild. Key threats include domain shift (Reddit vs. everyday text; recording booth vs. phone mic), speaker overlap in the acted corpus (if any), and threshold portability on text (calibrated to training priors). These are mitigated by reporting both formal and informal results, by keeping artifacts versioned and auditable, and by clearly separating probability estimation from decision calibration.

5.8 Summary

The tests reveal clear patterns in how the two emotion recognition systems behave across formal and informal environments. The audio model achieves 0.833 accuracy and 0.829 macro-F1 on acted speech, performing strongly in categories such as angry, calm, disgust, and surprised, while happy and sad remain weaker due to limited recall (see Fig. 5.4). The text model, using per-class thresholds, reaches 0.566 micro-F1 and 0.521 macro-F1, excelling on gratitude and amusement but struggling with diffuse or overlapping emotions in the long-tail distribution (see Fig. 5.3 and `per_class_metrics_csv` on GitHub).

When transferred from controlled datasets to everyday usage, the text model remains relatively stable across casual inputs, whereas the audio model drops to approximately the mid-60 % range—an expected outcome of domain shift from acted to spontaneous speech. Despite this, the prototype maintains low latency during usage, with an average end-to-end processing time of around 458 ms for text and 500 ms for audio on the local setup. These values demonstrate the efficiency of compact, well-calibrated models and lean pre/post-processing pipelines, ensuring smooth, near-real-time interaction.

Overall, these results highlight the complementary strengths of both modalities: text models offer robustness to informal inputs, while audio models provide better predictions in structured conditions. The observed constraints motivate future work toward speaker-disjoint training and augmentation strategies for speech emotion recognition, alongside continued calibration and imbalance mitigation for multi-label text classification. In the long term, expanding to a tri-modal framework could allow each modality to contribute what it captures best, improving both coverage and reliability across real-world scenarios.

6 DISCUSSION AND CONCLUSION

This final chapter interprets the results and derives practical conclusion. It explains why the audio model excels on acted benchmarks but getting less accurate in real use, why text remains comparatively stable yet needs threshold-aware calibration for fine-grained labels, and how a conceptual mapping enables qualitative cross-modal discussion. Additionally, this chapter assesses validity and ethics (domain shift, dataset bias, privacy), outlines limitations (speaker-overlap risk, threshold drift), and proposes focused next steps such as spontaneous speech data, speaker-disjoint splits, automated recalibration, on-device optimization and others. The chapter closes by arguing for modality-aware integration of these models into larger systems where each modality contributes its strengths.

6.1 Interpreting the findings

This thesis compared two modality-appropriate baselines for emotion recognition: a multi-label text classifier (BERT fine-tuned on GoEmotions) and a single-label speech classifier (a compact CRDNN trained with SpeechBrain on acted speech). On formal test splits, speech achieved 0.833 accuracy and 0.829 macro-F1, while text reached 0.566 micro-F1 and 0.521 macro-F1 on $\sim 5.4\text{k}$ samples. In informal mobile trials, the speech pathway dropped to ~ 0.64 accuracy—an expected domain shift from acted recording conditions to real-world phone audio—whereas the text pathway remained comparatively stable but showed recurrent confusions, especially Neutral vs. low-arousal content and difficulty with Disgust. Taken together, the results affirm that each modality excels under different constraints: audio captures prosodic cues that strengthen decisions for anger, disgust, and surprise; text captures lexicalized affect such as gratitude, amusement, but struggles on subtle, context-dependent states (e.g., realization, disappointment) where annotation sparsity and semantics make boundary drawing hard (Demszky et al., 2020).

6.2 Where errors come from

Errors reflect both data design and decision rules. GoEmotions’ fine-grained multi-label taxonomy exposes many tail classes; even with class weighting and tuned thresholds, sparse or diffuse labels remain challenging (Demszky et al., 2020). The audio corpus, being acted, emphasizes clean, high-contrast prosody; in everyday speech, low-energy contours cause sad to bleed into neutral/calm, and acted happy does not always match spontaneous delivery. Methodologically, moving from a fixed 0.50 cut-off to per-class thresholds for text reduced false multi-labeling and improved balance, validating calibration as part of the modeling pipeline. The CRDNN with softmax/NLL is appropriately strong on acted data (Ravanelli et al., 2021), but without augmentation, speaker-disjoint splits, or adaptation, robustness in the wild is limited. Finally, the many-to-one mapping used for cross-modal discussion enables qualitative comparison yet necessarily collapses nuance; a text prediction can be correct at fine grain and still disagree with a coarse audio category.

6.3 Validity, reliability, and ethics

Internal validity was supported by clean train/validation/test separation, threshold tuning on validation only, serialized inference configurations, and seeded/deterministic runs for audio following SpeechBrain recipe (Ravanelli et al., 2021). External validity remains conditional: emotion-stratified (rather than speaker-disjoint) splits may increase SER generalization; informal trials counterbalance this by revealing in-the-wild degradation. Text models trained on Reddit inherit domain and annotator biases that limit portability to other registers (Demszky et al., 2020). Construct validity is imperfect because rough audio categories and fine-grained text labels measure related but not identical constructs. To mitigate privacy risks, the prototype stores no raw audio and text, but keeps hashes of text and audio as meta-data (e.g for comparisons), and keeps analytics local; any real deployment would require explicit consent

and minimization. Observed gender-linked confusion patterns during trials are signals to audit, not definitive bias claims, particularly given known demographic artifacts in acted SER datasets (Puri et al., 2022).

6.4 Reflections and Future Work

In practice, the developed models demonstrate potential when integrated into larger, context-aware systems rather than used as standalone modules. In real-world applications, embedding the emotion recognition components within systems that maintain dialogue state, task awareness, or interaction history can improve predictions. The built-in feedback feature, which combines star ratings and user comments, provides valuable data for refining decision thresholds and selecting samples for retraining. In their current form, the models are best suited for low-risk, applications such as training voice actors, social-skills coaching, or affect-aware prompting. However, deployment in clinical or safety-critical uses would require new datasets, stricter validation, and governance.

Despite their promising results, several limitations affect interpretation. The use of different label granularities (fine-grained multi-label text vs. rough single-label audio) complicates direct comparison between modalities. The acted-speech dataset likely overestimates real-world generalization compared to spontaneous recordings, and emotion-stratified rather than speaker-disjoint splits introduce the risk of data leakage. For text, threshold calibration is tied to the priors of the dataset and may shift with domain balance change. Additionally, the prototype prioritizes clarity and privacy over user experience optimization, which may affect the quality of feedback collected during testing.

Looking ahead, the most promising future work directions are data-centric and model-centric. On the data side, collecting spontaneous speech in realistic environment, enforcing speaker-disjoint partitions, and applying augmentation techniques such as noise addition, room impulse

responses, or tempo and pitch variation would improve robustness. From a modeling standpoint, adding attention mechanisms or transformer-based acoustic encoders could improve speech emotion recognition, while hierarchical or label-graph heads may better capture dependencies among text-based emotion labels. Calibration could be automated through periodic threshold refresh based on user feedback, complemented by temperature scaling or focal loss to handle rare emotion classes. For multimodal expansion, a late-fusion baseline should precede end-to-end architectures capable of dynamically weighting modalities. Finally, on-device optimization via quantization or distillation, combined with a structured fairness inspection across speaker attributes and recording conditions, would support scalable deployment in real-world environments.

6.5 Conclusion

The project delivered a working, two-modal prototype with transparent, reproducible pipelines and a privacy-respecting feedback loop. Empirically, speech excels on acted benchmarks but degrades in the day to day scenarios; text is steadier across inputs but needs calibrated, class-aware decision rules to handle fine-grained, imbalanced labels. These findings support for modality-aware modeling, honest reporting across formal and informal contexts, and incremental integration into applications where each modality contributes its strengths—grounded in robust tooling (Devlin et al., 2019; Demszky et al., 2020; Ravanelli et al., 2021) and an explicit plan to close the lab-to-field gap (Puri et al., 2022).

REFERENCES

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT), 4171–4186.
<https://doi.org/10.18653/v1/N19-1423>
- Demszky, D., Movshovitz-Attias, D., Ko, J., Cowen, A., Nemade, G., & Ravi, S. (2020). GoEmotions: A dataset of fine-grained emotions. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 4040–4054.
<https://doi.org/10.18653/v1/2020.acl-main.372>
- Poria, S., Cambria, E., Hazarika, D., Majumder, N., Zadeh, A., & Morency, L.-P. (2018). Multimodal sentiment analysis: Addressing key issues and setting up the baselines. IEEE Intelligent Systems, 33(6), 17–25. <https://sentiment.net/multimodal-sentiment-analysis-key-issues-and-baselines.pdf>
- Zadeh, A., Liang, P. P., Poria, S., Cambria, E., & Morency, L.-P. (2018). Multimodal language analysis in the wild: CMU-MOSEI dataset and interpretable dynamic fusion graph. Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL), 2236–2246. <https://doi.org/10.18653/v1/P18-1208>
- Quan, W., Wang, Y., Chen, H., & Zhang, H. (2024). TCAN: Text-oriented cross-attention network for multimodal sentiment analysis. arXiv preprint, arXiv:2403.04545.
<https://arxiv.org/abs/2404.04545>

- Byun, S.-W. (2021). Multi-modal emotion recognition using speech features and text-embedding. <https://www.mdpi.com/2076-3417/11/17/7967>
- Ravanelli, M., Parcollet, T., Plantinga, P., Rouhe, A., Cornell, S., Nanchen, A., ... Bengio, Y. (2021). SpeechBrain: A general-purpose speech toolkit. arXiv preprint, arXiv:2106.04624. <https://arxiv.org/abs/2106.04624>
- Puri, S., Chandra, A., & Mukherjee, S. (2022). Detection of emotion of speech for RAVDESS audio using hybrid convolution neural network. Journal of Healthcare Engineering, 2022, 4857022. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8898841/>
- (Retracted; cited for dataset description consistent with the original RAVDESS release.)

APPENDIX: SOURCE CODE (GITHUB)

All the source code and implementation results and related files can be found on GitHub: https://github.com/Daniil669/Emotional_AI